

XRPL Validator — Daily Engineering Report • 2026-09-02

Compiled retrospectively on 2026-09-23 from the git history. Scope: XRPL work only — the git history of [xrpl-validator](#) and the XRPL Rust SDK, the XRP Ledger itself (amendments) and XRPL research.

Summary

- 32 commits (xrpl-validator 32)
- 1 test / vector commit

Engineering changes

- **fix(ledger): AccountDelete re-arms the destination's password change fee (finding 80)** (4bb3198 , xrpl-validator) DeleteAccount::doApply clears the destination's lsfPasswordSpent when it credits a positive balance (DeleteAccount.cpp:436-437), exactly as an XRP Payment does; we kept the bit. #106702154 67271C65: the destination held 0x20010000 and mainnet leaves 0x20000000. Pinned as accountdelete_password_spent_106702154 (red at HEAD~, green here).
- **fix(ledger): asfAllowTrustLineClawback needs an empty owner directory (finding 86)** (d180b37 , xrpl-validator) SetAccount refuses SetFlag 16 with tecNO_PERMISSION while lsfNoFreeze is set and with tecOWNERS while the owner directory holds anything (SetAccount.cpp:278-292); we set the bit regardless. #106703565 9709366F: an account owning 96 objects — mainnet tecOWNERS, we succeeded. Pinned as accountset_clawback_owners_106703565; the copied vector helpers now honour the bundle's recorded result instead of assuming tesSUCCESS.
- **fix(ledger): SignerListSet files its entries in account order (finding 83)** (6ae1dd1 , xrpl-validator) SetSignerList sorts the deserialized SignerEntries before writing the list (SetSignerList.cpp:66); we filed them in transaction order. #106703005 F62BBA1C. Pinned as signerlist_sorted_entries_106703005.
- **fix(ledger): the LP burn rounds the pool balance to nearest (finding 84)** (2ad147b , xrpl-validator) bump_lp_balance rounded to nearest only when minting and truncated when burning; the subtraction is the same Number arithmetic. #106702692 6F7C52C0 (tfWithdrawAll): 29320.99190061032 - 993.427226328693 rests at 28327.56467428163 on mainnet, ours ended ...62. Pinned as amm_withdraw_all_lp_rounding_106702692.
- **fix(ledger): AMMDeposit tfLimitLPToken prices the deposit against EPrice (finding 82)** (bf7ea3e , xrpl-validator) singleDepositEPrice (AMMDeposit.cpp): the full Amount is deposited at Equation 3's tokens when its effective price is within EPrice, otherwise the amount that lands exactly on EPrice is solved from the quadratic (asset rounded up, tokens rounded down, adjustAssetInByTokens after). The mode used to fall through to a 1e7-token placeholder. #106702459 F648B73A: 11.172545 XRP at EPrice 4 drops mints 3320034.6897731 ...
- **fix(ledger): escrow and channel creation reserve for the new object (finding 85)** (46c90df , xrpl-validator) EscrowCreate judges the post-fee balance against accountReserve(OwnerCount + 1) before anything else — tecINSUFFICIENT_RESERVE for XRP and token escrows alike, then tecUNFUNDED when an XRP amount does not fit above the reserve (Escrow.cpp:496-506); PayChanCreate's preclaim applies the same two tests

(PayChan.cpp:205-218). We had no reserve test on either path. #106703533 F84D3EB3 and #106703535 F44C919F: 1000 drops ...

- **fix(ledger): strand remainders are IOUAmounts and the flow ends on the gross budget (finding 81)** (abb13c0, xrpl-validator) rippled's remainders live in IOUAmount: Number-normalized to 16 digits and zero below an exponent of -96, and the payment loop runs while ONE gross remainder (remainingIn != f.in) stays positive. Our walk kept every digit and the payment driver broke on a net twin that can carry a one-ULP residue after the strand consumed the whole SendMax. #106703062 AC58204A: 1.335056517376953e-6 BTC spent in full, net residue ...
- **feat(harness): bundles carry the pool's owner objects for AMM transactions** (e64db88, xrpl-validator) fixAMMv1_1's isOnlyLiquidityProvider walks the AMM account's owner directory counting LP lines; with the other LPs' lines absent a withdraw probe declared the withdrawer the sole LP and refused with tecAMM_INVALID_TOKENS (#106702692 6F7C52C0, live tesSUCCESS). AMM transactions now pull the pool's account_objects and owner directory.
- **perf(node): state_replay keeps raw leaves and decodes on read** (9f513a9, xrpl-validator) A full mainnet state replay held ~32 GB of engine JSON (19.8M objects) beside xrpld's 13 GB on the same host, thrashing swap and knocking the upstream into tracking. Leaves now hold the serialized blob and LedgerState::read_json decodes through an optional leaf_decoder hook (installed by state_replay: decode → hexify addresses → index). JSON leaves (leading { }) still read as stored, so the engine and every ...
- **fix(ledger): AMMDeposit tfOneAssetLPToken derives the asset from the tokens (finding 87)** (1e7c9e9, xrpl-validator) rippled's singleDepositTokens (AMMDeposit.cpp) sizes a tfOneAssetLPToken deposit from the requested LPTokenOut: adjust the tokens to the pool's precision, compute the asset in via ammAssetIn for exactly those tokens, and fail with tecAMM_FAILED when that exceeds the offered Amount. Ours sized from the Amount and applied the token cap afterwards, minting a different LP balance and pool root than mainnet.
- **fix(ledger): AMM maxOffer's 99% cap multiplies nearest, converts down (finding 88)** (5557187, xrpl-validator) AMMLiquidity::maxOffer sizes the single-path, no-CLOB pool pass as toAmount(out * Number{99, -2}, Downward): the multiply runs in Number's default nearest mode and only the conversion rounds down. consume rounded the product down as well, one ulp short whenever the cap binds — a payment draining more than 99% of a pool — and swapAssetOut of the smaller out then moved the in side by ~3e-13. max_offer_amounts ...
- **fix(ledger): findings 81b, 89, 90, 91 — fill remainders, fee-leg spend, QF limit, saved-in fold** (e278926, xrpl-validator) Four engine findings from the 09-02 drill, gated together by HIST60 and WIN98 (the vectors pin each one separately).
- **fix(ledger): CheckCreate refuses a frozen SendMax (finding 92)** (7395402, xrpl-validator) CheckCreate::preclaim (rippled CheckCreate.cpp, ungated) rejects a non-XRP SendMax with tecFROZEN when the currency is globally frozen, when the issuer froze the source's trust line (the issuer's side bit), or when the destination froze its own line (the destination's side bit); a party without a line passes. We had no freeze block and created the Check and both directory entries where mainnet claims only the fee.
- **fix(ledger): an in-limited bridged slice floors the XRP mid-leg for a book leg too (finding 93)** (2610407, xrpl-validator) The bridged walk rounded the XRP mid-leg up unless leg A was a pool that exhausted the taker's in (finding 51) or the leg-B maker's funds bound the slice (finding 62). rippled's forward pass is in-limited whenever the slice takes the taker's whole remaining in — limitStepIn → ceilInStrict(..., roundUp = false) → divRoundStrict(limit, rate, XRP, false) — for any offer type, so a BOOK leg A floors as well.

- **fix(node): ServerVersion and amendment votes ride the voting ledger only** (cece712 , xrpl-validator) rippled attaches its server version and its amendment (and fee) votes to the validation of the ledger directly PRECEDING a flag ledger — `Ledger::isVotingLedger()`, `seq % 256 == 255` (`RCLConsensus.cpp validate()`). We attached `ServerVersion` to every validation since 2026-08-29 and amendment votes to the flag ledger itself (`seq % 256 == 0`), so observers classified both as off-cadence and discarded them.
- **fix(scripts): fetch_tx_bundle replays directory pages created earlier in the ledger** (a76193e , xrpl-validator) A page `CREATED` by an earlier tx of the same ledger was seated from that tx's `NewFields` (or this tx's `FinalFields`), which never carry `sfIndexes` (`sMD_Never`) — an `EMPTY` page. q63 509460161CB0@106711585 crossed nothing on it and reported the maker's three objects `NOT WRITTEN`; with the page's one entry replayed the tx is byte-exact. Route such pages through `ensure_page` (entries replayed from empty through the earlier ...
- **fix(ledger): direct IOU payments size the fee step with rippled's mulRatio (finding 96)** (d4a4421 , xrpl-validator) `apply_iou_direct` sized the transfer-fee step with `me_muldiv` — a plain ceil for the reverse pass and a plain floor for delivery. rippled's `DirectStepI` runs `IOUAmount mulRatio` on both passes (`DirectStep.cpp:537, :646`): the quotient is carried at 18 digits, the `IOUAmount` constructor normalizes to 16 digits at `Number`'s half-even `NEAREST`, and only then does a surviving remainder add one ulp for `roundUp`. For a ...
- **fix(ledger): tick-size TakerPays multiply rounds half-even to drops (finding 95)** (87383f0 , xrpl-validator) `st_multiply`'s XRP side still carried the legacy `STAmount` form `muldiv(v1, v2, 10^14) + 7`, which is dead under `STNumberSwitchover`: rippled runs `Number{v1} * Number{v2}` then `XRPAmount{Number}` (`STAmount.cpp:1416`, `canonicalize :880`) — the product folded half-even to 16 digits, then half-even to integer drops ("round to nearest, even on tie", `Number.h:395`). The `+7` breaks every exact half-drop tie upward.
- **fix(ledger): NFT royalty is waived when the buyer or the seller is the issuer (finding 94)** (20afcdb , xrpl-validator) rippled pays the `NFToken` transfer-fee cut only when `seller != issuer && buyer != issuer` (`NFTokenAcceptOffer.cpp:423 direct, :542 brokered`). All three of our payout helpers tested the seller alone, so an issuer buying its own token back was charged the royalty against the seller and credited with it: #106711435 DC521081D420 (`brokered`, `TransferFee 5 %`) moved 500000 drops from the seller to the buyer; #106711449 ...
- **fix(ledger): bridge-attempt direct head floors a maker-funds-bound fill (finding 97)** (2882024 , xrpl-validator) The third fill site in `cross_engine_to_net` — the direct-book head that competes inside a bridge attempt — priced every partial fill with `mul_round16_up`, the `limitOut(..., roundUp = true)` ceil that belongs to a `TAKER`-bound partial (`BookStep.cpp:675`). When the `MAKER`'s funds bind, rippled runs the funds clamp instead: `limitOut(..., roundUp = false)` (:790), `mulRoundStrict` under the `Number` guard at `TowardsZero` — a ...
- **fix(ledger): NFTokenAcceptOffer refuses own and looped offers (finding 100)** (e713882 , xrpl-validator) rippled's preclaim has three `tecCANT_ACCEPT_OWN_NFTOKEN_OFFER` rules we never ported: in `brokered` mode the buy and sell offers may not belong to one account — "the two offers may not form a loop; a broker may not sell the token to the current owner" (`NFTokenAcceptOffer.cpp:105-108`) — and in either mode an account can't accept an offer it placed itself (:165-167 for a buy offer, :224-226 for a sell offer).
- **fix(ledger): the pool's spot beating a fee-band tip admits the AMM tail (finding 99)** (5e036e6 , xrpl-validator) An offer crossing's tail turn refused the pool whenever the residual order book tip sat between the net limit and the transfer-rate-inflated one — the #106225714 shape, where the `CLOB` tip IS the tip, its `qualityUpperBound` carries the input fee and misses the limit. But rippled's `tip()` first anchors the AMM offer on the `LOB` quality (`BookStep.cpp:938-948`): when the pool's spot beats the tip that synthetic offer ...
- **fix(ledger): a payment's intermediate legs never land on the sender's line (finding 98)** (989927a , xrpl-validator) rippled's strand hands a `BookStep`'s output straight to the next step — the issuer's `DirectStep` or the next

book's makers — and the sender's own line in the pass-through currency is never touched. Our hop chain settled every hop against the taker: a hop that is not the first parked its IN on the sender, a hop that is not the last parked its OUT there, and the next hop took it back. Inert on an existing line; on a ...

- **fix(ledger): AMM tail admission judges the anchored pool offer, not the spot (finding 99, revised)** (f c83c7e , xrpl-validator) Finding 99 admitted the pool whenever its spot beat an order-book tip inside the fee band. Five hundred ledgers later the same account and book showed the rule was too loose: #106715477 01019A7D3E59 and #106715482 52CFB1FDE091 have the spot better than the tip too, yet mainnet rests both offers — "All strands dry" in rippled's trace.
- **fix(ledger): a level's pool turn runs before its offers are swept (finding 101)** (b2f1bc7 , xrpl-validator) rippled's forEachOffer tries the AMM at the tip's quality BEFORE visiting the level's CLOB offers (BookStep.cpp:837-846): `tryAMM(offers.tip().quality())` executes the anchored pool offer, and when that covers the remaining want `execOffer` returns false and the tip is never reached — `limitSelfCrossQuality` never runs on it. Our beyond-limit sweep reaped the level first.
- **fix(ledger): a tfTwoAsset withdraw that fits neither ordering is tecAMM_FAILED (finding 102)** (6527875 , xrpl-validator) `equal_withdraw_limit` already mirrored rippled's `equalWithdrawLimit` — lead with Amount and derive Amount2, else lead with Amount2 and derive Amount — but when NEITHER ordering fit both maxima the caller fell through to the verbatim move and paid both amounts in full. Under `fixAMMv1_3` rippled returns `tecAMM_FAILED` there (AMMWithdraw.cpp:949).
- **fix(ledger): AMMDeposit runs requireAuth and checkFrozen on both pool assets (finding 104)** (0bd064e , xrpl-validator) rippled's `AMMDeposit::preclaim` (featureAMMClawback, AMMDeposit.cpp:255-283) passes BOTH pool assets through `requireAuth(WeakAuth)` and `checkFrozen` for the depositor before any funding arithmetic, then the deposited amounts through the STRONG check (:297). `requireAuth` (RippleStateHelpers.cpp): an issuer with `lsfRequireAuth` and no line is `tecNO_LINE`, a line without this side's auth bit `tecNO_AUTH`; a strong check with ...
- **fix(ledger): a book behind an issuer-NoRipple sender line is tecPATH_DRY (finding 105)** (878aaa9 , xrpl-validator) rippled's `BookStep::check` refuses a book that follows a `DirectStepI` when the line between that step's source and the book's in-issuer is missing (`terNO_LINE`) or carries `NoRipple` on the ISSUER's side (`terNO_RIPPLE`); `Payment` turns the `ter` into `tecPATH_DRY`. Every strand we model opens with the sender -> spend-issuer step and then its first book, so the flag on the issuer's side of the sender's spend line refuses the ...
- **fix(ledger): a flow's pools go silent after 30 AMM iterations (finding 107)** (17eaaac , xrpl-validator) rippled's `AMMContext` keeps ONE iteration counter per flow: `AMMOffer::consume` sets `ammUsed_` in either offer mode (fib slice or single-path), `flow()` calls `ammContext.update()` once per winning iteration, and `AMMLiquidity::getOffer` answers `nullopt` for the rest of the flow once `ammIters_ >= kMaxIterations` (30). Our `AmmFib` counted only fib-mode rounds and never capped, and the fib table stopped at 16 entries.
- **fix(ledger): a strand's pool bound prices the slice the pass will consume (finding 109)** (4c0a834 , xrpl-validator) rippled's `AMMLiquidity` keeps `initialBalances_` from the flow's start and generateFibSeqOffer seeds every fib slice from them; `qualityUpperBound` and the pass see the same offer. Our consume path records that origin in `AmmFib.init` at a pool's first use, but the strand upper bound sized its slice from the pool's CURRENT balances, so after a few rounds the bound and the real slice drifted $\sim 1e-4$ apart and a near-tie ...
- **fix(ledger): a deposit whose adjusted asset-in reaches zero mints no tokens (finding 108)** (b884eeb , xrpl-validator) rippled's `adjustAssetInByTokens` (`fixAMMv1_3`) recomputes the asset-in for the tokens a deposit earns; when that rounded-up amount overshoots the deposit, it shrinks the amount by the overshoot, prices the tokens for

the shrunken amount and re-adjusts — and a zero (or negative) result is the answer: AMMDeposit refuses tokensAdj == 0 with tecAMM_INVALID_TOKENS, and deposit() refuses a non-positive actual the same way. ...

Tests and campaigns

- **test(node): PROBE_LIST dumps every write of a bundle apply** (2727985 , xrpl-validator)

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **Oracle-Bounded Concentrated AMM Liquidity** (amm-defi) Add a new AMMLiquidityPosition ledger object (owner-reserved, keylet::ammPosition(ammID, account, seq)) carrying sfTickLower/sfTickUpper price-tick SF
- **Shielded EscrowFinish for Confidential Transfers** (privacy) Add an lsfConfidential flag and sfAmountCommitment (a Pedersen commitment, reusing ConfidentialTransfer's commitment scheme) to Escrow objects created
- **Byte-Weighted Owner Reserves** (fees) Replace the flat per-object OwnerCount increment with sfReserveUnits, a deterministic ceil(serializedSize/128) computed at object creation for RippleS

Appendix — commits

Time	Repo	Commit	Subject
00:27	xrpl-validator	4bb3198	fix(ledger): AccountDelete re-arms the destination's password change fee (finding 80)
00:28	xrpl-validator	d180b37	fix(ledger): asfAllowTrustLineClawback needs an empty owner directory (finding 86)
00:30	xrpl-validator	6ae1dd1	fix(ledger): SignerListSet files its entries in account order (finding 83)
00:34	xrpl-validator	2ad147b	fix(ledger): the LP burn rounds the pool balance to nearest (finding 84)
00:37	xrpl-validator	b7f7ea3e	fix(ledger): AMMDeposit tfLimitLPToken prices the deposit against EPrice (finding 82)
00:40	xrpl-validator	46c90df	fix(ledger): escrow and channel creation reserve for the new object (finding 85)
00:41	xrpl-validator	abb13c0	fix(ledger): strand remainders are IOUAmounts and the flow ends on the gross budget (finding 81)
00:41	xrpl-validator	e64db88	feat(harness): bundles carry the pool's owner objects for AMM transactions
02:21	xrpl-validator	9f513a9	perf(node): state_replay keeps raw leaves and decodes on read
02:21	xrpl-validator	1e7c9e9	fix(ledger): AMMDeposit tfOneAssetLPToken derives the asset from the tokens (finding 87)
02:39	xrpl-validator	5557187	fix(ledger): AMM maxOffer's 99% cap multiplies nearest, converts down (finding 88)
04:56	xrpl-validator	e278926	fix(ledger): findings 81b, 89, 90, 91 — fill remainders, fee-leg spend, QF limit, saved-in fold
05:17	xrpl-validator	7395402	fix(ledger): CheckCreate refuses a frozen SendMax (finding 92)
06:03	xrpl-validator	2610407	fix(ledger): an in-limited bridged slice floors the XRP mid-leg for a book leg too (finding 93)
07:25	xrpl-validator	cece712	fix(node): ServerVersion and amendment votes ride the voting ledger only
09:35	xrpl-validator	a76193e	fix(scripts): fetch_tx_bundle replays directory pages created earlier in the ledger
09:35	xrpl-validator	d4a4421	fix(ledger): direct IOU payments size the fee step with rippled's mulRatio (finding 96)
09:35	xrpl-validator	87383f0	fix(ledger): tick-size TakerPays multiply rounds half-even to drops (finding 95)

Time	Repo	Commit	Subject
09:35	xrpl-validator	20afcdb	fix(ledger): NFT royalty is waived when the buyer or the seller is the issuer (finding 94)
10:28	xrpl-validator	2882024	fix(ledger): bridge-attempt direct head floors a maker-funds-bound fill (finding 97)
12:47	xrpl-validator	e713882	fix(ledger): NFTokenAcceptOffer refuses own and looped offers (finding 100)
12:47	xrpl-validator	5e036e6	fix(ledger): the pool's spot beating a fee-band tip admits the AMM tail (finding 99)
12:47	xrpl-validator	2727985	test(node): PROBE_LIST dumps every write of a bundle apply
12:47	xrpl-validator	989927a	fix(ledger): a payment's intermediate legs never land on the sender's line (finding 98)
13:48	xrpl-validator	fc83c7e	fix(ledger): AMM tail admission judges the anchored pool offer, not the spot (finding 99, revised)
20:07	xrpl-validator	b2f1bc7	fix(ledger): a level's pool turn runs before its offers are swept (finding 101)
20:07	xrpl-validator	6527875	fix(ledger): a tfTwoAsset withdraw that fits neither ordering is tecAMM_FAILED (finding 102)
20:07	xrpl-validator	0bd064e	fix(ledger): AMMDeposit runs requireAuth and checkFrozen on both pool assets (finding 104)
20:55	xrpl-validator	878aaa9	fix(ledger): a book behind an issuer-NoRipple sender line is tecPATH_DRY (finding 105)
22:53	xrpl-validator	17eaaac	fix(ledger): a flow's pools go silent after 30 AMM iterations (finding 107)
23:43	xrpl-validator	4c0a834	fix(ledger): a strand's pool bound prices the slice the pass will consume (finding 109)
23:43	xrpl-validator	b884eeb	fix(ledger): a deposit whose adjusted asset-in reaches zero mints no tokens (finding 108)